

Разработка REST API и CRUD-операций

Создаём полноценный API для ресурсов интернет-магазина

Результаты занятия

- ✓ Объяснять назначение REST API
- ✓ Проектировать URL вокруг ресурсов приложения
- ✓ Связывать CRUD-операции с HTTP-методами
- ✓ Использовать коллекции и отдельные ресурсы
- ✓ Реализовывать CRUD через routes, controllers и services
- ✓ Возвращать подходящие HTTP-коды состояния
- ✓ Проверять REST API через HTTP-клиент

Учебные вопросы

REST API и ресурсы

Как проектировать API
вокруг сущностей
приложения

Проектирование endpoints

Коллекции, отдельные
ресурсы и понятные
URL

Ответы REST API

HTTP-коды, JSON и
проверка полного
CRUD-цикла

CRUD и HTTP- методы

Create, Read, Update,
Delete и их связь с HTTP

Реализация CRUD в Express

Routes, controllers и
services для Product API



Учебный вопрос 1. REST API и ресурсы

API как интерфейс между frontend и серверной частью приложения

Что такое API

- API — программный интерфейс, через который одна часть системы может обращаться к функциям и данным другой части системы.
- Frontend не должен напрямую обращаться к серверным файлам или базе данных.
- Клиент отправляет запросы к определённым endpoints.
- Backend обрабатывает запрос и возвращает ответ.
- Для браузерных приложений обмен часто выполняется по HTTP.
- Данные между frontend и backend обычно передаются в JSON.

Что такое REST API

- REST API — HTTP API, организованный вокруг ресурсов и использующий стандартную семантику HTTP для выполнения операций над ними.
- API рассматривает данные приложения как ресурсы.
- Ресурс имеет понятный URL.
- HTTP-метод показывает требуемую операцию.
- Сервер возвращает HTTP-код и представление результата.
- Один и тот же ресурс используется для нескольких операций.

Главная идея

В REST API действие обычно выражается HTTP-методом, а URL обозначает ресурс.

Ресурсы интернет-магазина

● Products

Товары магазина: название, цена, категория, наличие.

● Categories

Категории, по которым организуются товары.

● Orders

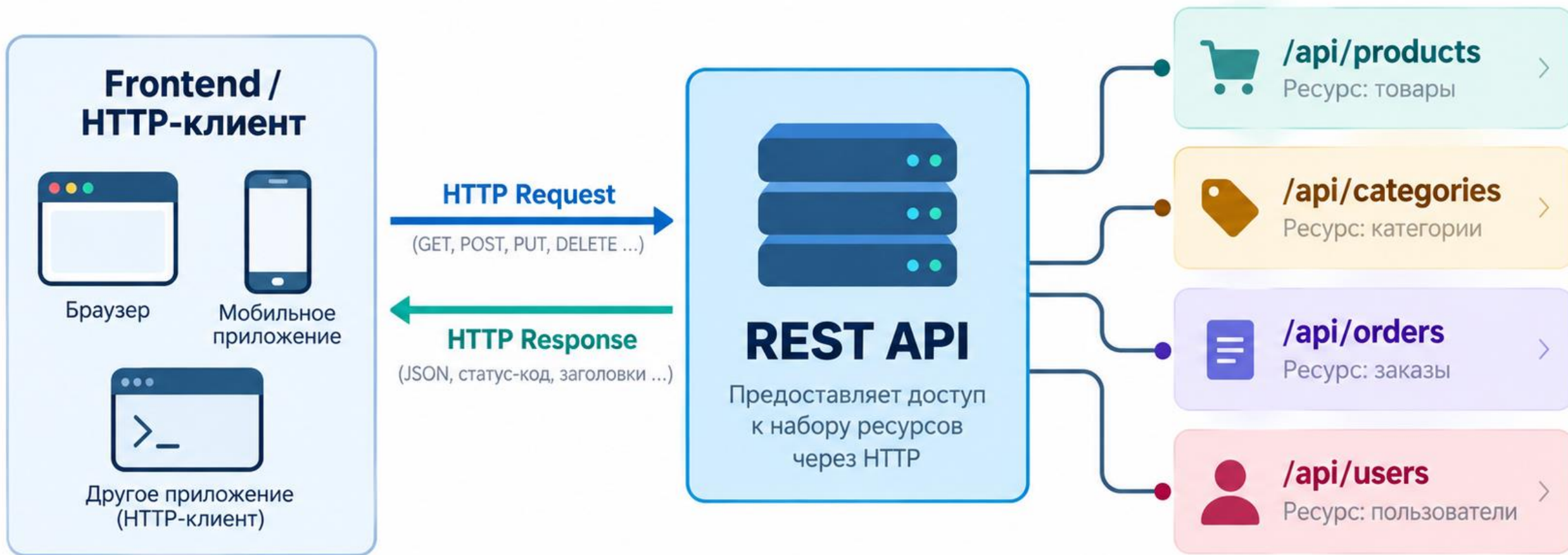
Заказы покупателей и информация об их состоянии.

● Users

Пользователи приложения и связанные с ними данные.

REST API как набор ресурсов

Один API — множество ресурсов. Доступ к ресурсам через HTTP.



URL обозначает ресурс

Например: `/api/products` — это ресурс «товары»



HTTP-метод определяет действие

Например: GET — получить, POST — создать, PUT — обновить, DELETE — удалить

URL как действие и URL как ресурс

Менее удачный подход

- GET /getProducts
- GET /getProductById/15
- POST /createProduct
- POST /updateProduct/15
- POST /deleteProduct/15

Ресурсный подход

- GET /api/products
- GET /api/products/15
- POST /api/products
- PATCH /api/products/15
- DELETE /api/products/15

Почему в URL используют существительные

- URL обозначает ресурс, с которым работает клиент.
- HTTP-метод уже описывает действие над ресурсом.
- `products` обозначает коллекцию товаров.
- `products/15` обозначает конкретный товар.
- Такой API легче читать и использовать.

Главная идея

GET, POST, PATCH и DELETE уже говорят, что нужно сделать, поэтому `create` или `delete` в URL обычно не нужны.



Учебный вопрос 2. CRUD и HTTP-методы

Четыре базовые операции над данными

CRUD

- CRUD описывает базовые операции с данными независимо от конкретной технологии.
- Create — создание нового ресурса.
- Read — получение одного или нескольких ресурсов.
- Update — изменение существующего ресурса.
- Delete — удаление существующего ресурса.

Простые
действия —
большие
возможности

CRUD и HTTP-методы

Один ресурс — четыре базовые операции

HTTP — это способ
работы с ресурсами
на сервере

Create



POST

создать ресурс

```
POST /products
{
  "name": "Ноутбук",
  "price": 59900
}
```

Read



GET

получить ресурс

```
GET /products/1
// Ответ сервера:
{
  "id": 1,
  "name": "Ноутбук",
  "price": 59900
}
```

Product



```
id: 1
name: "Ноутбук"
price: 59900
stock: 10
....
```

Product — это ресурс
(например, товар
в интернет-магазине)

Update



PATCH / PUT

изменить ресурс

```
PATCH /products/1
{
  "price": 54900
}
или
PUT /products/1
{
  "name": "Ноутбук",
  "price": 54900,
  "stock": 5
}
```

Delete



DELETE

удалить ресурс

```
DELETE /products/1
// Ресурс удален
// (ответ: 204 No Content)
```



CRUD + HTTP = простой и понятный интерфейс взаимодействия с ресурсами

Те же принципы работают
для любых ресурсов

CRUD для ресурса products

CRUD	HTTP	Endpoint	Назначение
Create	POST	/api/products	Создать товар
Read	GET	/api/products	Получить список товаров
Read	GET	/api/products/:id	Получить один товар
Update	PATCH	/api/products/:id	Частично изменить товар
Delete	DELETE	/api/products/:id	Удалить товар

Read: получение данных

- GET используется для чтения ресурса.
- GET /api/products возвращает коллекцию товаров.
- GET /api/products/15 возвращает один товар.
- Получение данных не должно само по себе изменять ресурс.
- Для выбора конкретного объекта используется параметр маршрута.

Получение одного товара

Запрос: GET /api/products/15

```
{  
  "id": 15,  
  "name": "SSD 1TB",  
  "price": 45000,  
  "inStock": true  
}
```

Пояснение

- Клиент запрашивает конкретный ресурс products/15. При успешном поиске сервер возвращает товар в JSON.

Create: создание ресурса

- POST обычно используется для создания нового элемента коллекции.
- URL указывает коллекцию `/api/products`.
- Данные нового товара передаются в `req.body`.
- Server создаёт новый `id`.
- Созданный объект можно вернуть клиенту.
- Для успешного создания обычно используется статус 201 Created.

Создание товара

Запрос: POST /api/products

```
{  
  "name": "Mechanical Keyboard",  
  "price": 32000,  
  "category": "Accessories",  
  "inStock": true  
}
```

```
{  
  "id": 16,  
  "name": "Mechanical Keyboard",  
  "price": 32000,  
  "category": "Accessories",  
  "inStock": true  
}
```

Пояснение

- Клиент отправляет данные нового товара, а сервер создаёт ресурс и возвращает его с назначенным id.

PUT и PATCH

PUT

- Ориентирован на замену представления ресурса
- Клиент передаёт новое состояние ресурса
- Используется для полного обновления
- PUT является идемпотентным

PATCH

- Применяет частичные изменения
- Можно передать только изменяемые поля
- Удобен для форм редактирования
- В учебном Product API используем PATCH

Простые
правила —
большие
возможности

POST, GET, PATCH и DELETE для одного ресурса

URL остаётся ресурсным, действие задаётся методом HTTP

Один и тот же URL,
разные операции!

POST /api/products

создаёт объект



Создаёт новый
продукт в коллекции

```
// Тело запроса (JSON)
{
  "name": "Ноутбук",
  "price": 59900,
  "stock": 10
}
```

Ответ (201 Created)

```
{
  "id": 16,
  "name": "Ноутбук",
  "price": 59900,
  "stock": 10
}
```

URL описывает
ресурс, а не действие

/api/products



Ресурс
(коллекция продуктов)

GET /api/products/15

читает объект



Получает данные
о продукте с id = 15

Ответ (200 OK)

```
{
  "id": 15,
  "name": "Смартфон",
  "price": 29990,
  "stock": 25
}
```

PATCH /api/products/15

изменяет объект



Обновляет часть
данных продукта

```
// Тело запроса (JSON)
{
  "price": 25990
}
```

Было: "price": 29990
Стало: "price": 25990

Ответ (200 OK)

```
{
  "id": 15,
  "name": "Смартфон",
  "price": 25990,
  "stock": 25
}
```

Тот же ресурс.
Разные методы.
Разный результат.

DELETE /api/products/15

удаляет объект



Удаляет продукт
с id = 15

Ответ (204 No Content)

// Тело ответа пустое
(ресурс удалён)



Один ресурс — разные операции

Метод HTTP определяет действие

Чистый API —
понятный API!

Частичное изменение товара

Запрос: PATCH /api/products/15

```
{  
  "price": 42000,  
  "inStock": false  
}
```

```
{  
  "id": 15,  
  "name": "SSD 1TB",  
  "price": 42000,  
  "inStock": false  
}
```

Пояснение

- PATCH позволяет изменить только переданные свойства, не отправляя заново весь объект.

Delete: удаление ресурса

- DELETE используется для удаления выбранного ресурса.
- Идентификатор обычно находится в URL.
- DELETE /api/products/15 означает удаление товара с id 15.
- Сервер сначала должен определить, существует ли такой ресурс.
- После удаления можно вернуть результат операции или ответ без тела.

Удаление товара

Запрос: DELETE /api/products/15

```
{  
  "id": 15,  
  "name": "SSD 1TB",  
  "price": 42000,  
  "inStock": false  
}
```

Пояснение

- В учебном API сервер возвращает удалённый объект. Другой распространённый вариант успешного удаления — 204 No Content без тела ответа.



Учебный вопрос 3. Проектирование endpoints

Коллекции, отдельные ресурсы и единая структура URL

Коллекция и отдельный ресурс

Коллекция

- /api/products
- Обозначает набор товаров
- GET получает список
- POST добавляет новый элемент

Отдельный ресурс

- /api/products/:id
- Обозначает конкретный товар
- GET получает один объект
- PATCH изменяет объект
- DELETE удаляет объект

Идентификатор ресурса

- Каждый товар должен иметь уникальный id.
- id позволяет обратиться к конкретному ресурсу.
- В URL используется динамический сегмент :id.
- Express помещает фактическое значение в req.params.id.
- Перед поиском строковое значение можно преобразовать в Number.

Маршруты Product API

```
router.get('/', getProducts);  
router.get('/:id', getProductById);  
router.post('/', createProduct);  
router.patch('/:id', updateProduct);  
router.delete('/:id', deleteProduct);
```

Что важно

- Один Router обслуживает ресурс products.
- Путь / работает с коллекцией.
- Путь /:id работает с конкретным товаром.
- HTTP-метод определяет нужную CRUD-операцию.
- Route передаёт обработку соответствующему controller.

REST API нескольких ресурсов

- `/api/products` — товары
- `/api/categories` — категории
- `/api/orders` — заказы
- `/api/users` — пользователи
- Для каждого ресурса можно создать собственный Router.
- Внутри каждого Router используется одинаковая логика проектирования CRUD.

REST —
простые правила,
большие
возможности!

Коллекция и отдельный ресурс в REST API

Один стиль URL.
Разные задачи!

Один тип ресурса — два URL. Коллекция содержит много элементов, отдельный ресурс — один.



/api/products

Коллекция ресурсов

Набор товаров (несколько объектов)



Ноутбук
59 990 ₽ id: 1



Смартфон
29 990 ₽ id: 2



Наушники
9 990 ₽ id: 3

...



Смарт-часы
14 990 ₽ id: 4



Кроссовки
6 990 ₽ id: 5



Рюкзак
4 990 ₽ id: 6



GET

/api/products

Получить список товаров
Возвращает массив товаров
(коллекцию).



POST

/api/products

Создать новый товар
Добавляет новый элемент
в коллекцию.

От коллекции
к конкретному
товару!

Один из товаров
из коллекции



Ноутбук
59 990 ₽ id: 15

Тот же тип ресурса.
Просто указан
его идентификатор.



/api/products/15

Отдельный ресурс

Один конкретный товар (один объект)



Ноутбук "Pro"

id: 15
price: 59990
stock: 10
description: "Мощный ноутбук
для работы и учебы"
category: "Ноутбуки"

15 — идентификатор
конкретного товара
из коллекции



GET

/api/products/15

Получить один товар

Возвращает данные конкретного
товара.



PATCH

/api/products/15

Изменить товар

Обновляет указанные поля
конкретного товара.



DELETE

/api/products/15

Удалить товар

Удаляет конкретный товар
из коллекции.



Один URL для коллекции, другой — для конкретного ресурса

HTTP-метод определяет действие

Чистый API —
понятный код!



Учебный вопрос 4. Реализация CRUD в Express

Product API через routes, controllers и services

CRUD через **Route** → **Controller** → **Service**

Одни и те же слои обрабатывают все операции **CRUD** (**GET**, **POST**, **PATCH**, **DELETE**) для продуктов.

HTTP Requests

(клиент → сервер)

- GET** /api/products
Получить список товаров
- POST** /api/products
Создать новый товар
- PATCH** /api/products/15
Обновить товар (id = 15)
- DELETE** /api/products/15
Удалить товар (id = 15)

Product Router

Выбирает маршрут по HTTP-методу и URL.

Product Controller

Работает с req и res, вызывает service, формирует HTTP-ответ.

Product Service

Выполняет бизнес-логику и операции CRUD.

Источник данных: products[]

products[] = [

{ id: 1, name: "Ноутбук", price: 1200 },

{ id: 2, name: "Смартфон", price: 800 },

{ id: 15, name: "Наушники", price: 150 },
...
]

Хранит текущие данные товаров.

- 200 OK** (список товаров)
- 201 Created** (товар создан)
- 200 OK** (товар обновлён)
- 204 No Content** (товар удалён)

HTTP Response возвращается клиенту в зависимости от операции.

HTTP Response

(сервер → клиент)

Controller преобразует результат в JSON и устанавливает HTTP-статус (например, 200, 201, 404).

Роли слоёв:

Product Router

Выбирает маршрут по HTTP-методу и URL.

Product Controller

Работает с req и res, вызывает service, формирует HTTP-ответ.

Product Service

Выполняет бизнес-логику и операции CRUD.

Массив данных products[]

Хранит текущие данные товаров.

Начальные данные productService.js

```
let products = [  
  {  
    id: 1,  
    name: 'Keyboard',  
    price: 15000,  
    inStock: true  
  },  
  {  
    id: 2,  
    name: 'Mouse',  
    price: 8000,  
    inStock: true  
  }  
];
```

Read в Service

```
export function getAllProducts() {  
  return products;  
}  
  
export function findProductById(id) {  
  return products.find(  
    product => product.id === id  
  );  
}
```

Что важно

- getAllProducts возвращает коллекцию.
- findProductById ищет один объект.
- Service получает обычный id.
- Service не работает с req и res.

Create в Service

```
export function addProduct(data) {  
  const product = {  
    id: products.length + 1,  
    ...data  
  };  
  
  products.push(product);  
  return product;  
}
```

Что важно

- Service получает данные нового товара.
- Создаётся новый объект.
- Товар получает id.
- Объект добавляется в массив.
- Созданный товар возвращается controller.

Update в Service

```
export function updateProductById(id, changes) {  
  const product = products.find(  
    product => product.id === id  
  );  
  
  if (!product) return null;  
  
  Object.assign(product, changes);  
  return product;  
}
```

Что важно

- Сначала выполняется поиск товара.
- Если товар отсутствует, возвращается null.
- changes содержит свойства из тела PATCH-запроса.
- Object.assign изменяет только переданные свойства.
- Service возвращает обновлённый объект.

Delete в Service

```
export function deleteProductById(id) {  
  const index = products.findIndex(  
    product => product.id === id  
  );  
  
  if (index === -1) return null;  
  
  const [deletedProduct] = products.splice(index, 1);  
  return deletedProduct;  
}
```

Что важно

- `findIndex` определяет позицию товара.
- Значение `-1` означает, что товар не найден.
- `splice` удаляет элемент массива.
- Удалённый объект возвращается `controller`.

Controllers для чтения

```
export function getProducts(req, res) {
  res.json(getAllProducts());
}

export function getProductById(req, res) {
  const id = Number(req.params.id);
  const product = findProductById(id);

  if (!product) {
    return res.status(404).json({
      message: 'Product not found'
    });
  }

  res.json(product);
}
```

Что важно

- Controller получает HTTP-запрос.
- Поиск данных выполняет service.
- Для существующего товара возвращается JSON.
- Для отсутствующего товара возвращается 404.

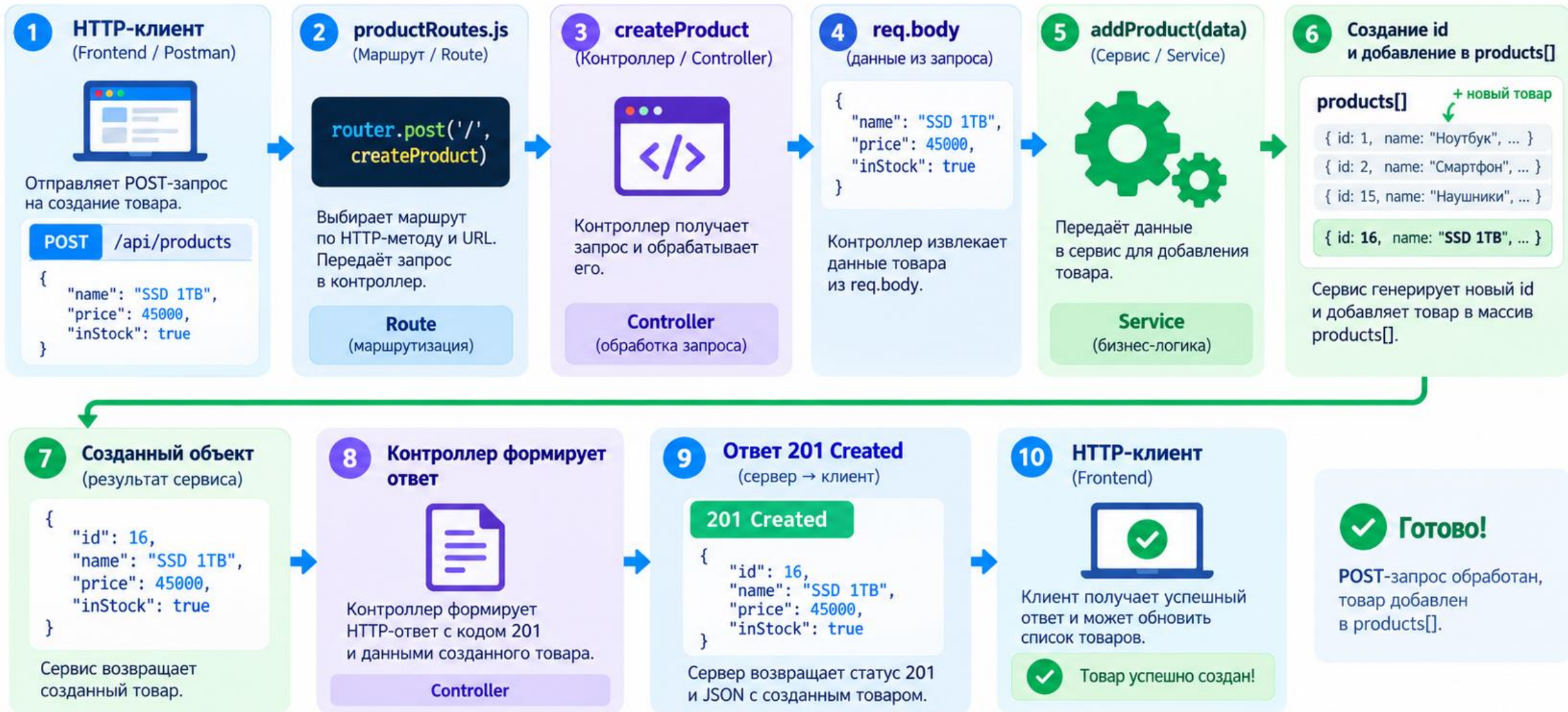
Controller создания

```
export function createProduct(req, res) {  
  const product = addProduct(req.body);  
  
  res.status(201).json(product);  
}
```

Что важно

- Данные поступают через req.body.
- Controller передаёт их в service.
- Service возвращает созданный объект.
- Controller отвечает статусом 201 Created.

Путь **POST**-запроса при создании товара



Роли слоёв в приложении:



Route (маршрут)

Сопоставляет HTTP-метод и URL, передаёт запрос в контроллер.



Controller (контроллер)

Получает запрос, извлекает данные, вызывает сервис, формирует ответ.



Service (сервис)

Реализует бизнес-логику: создание id, добавление товара в products[].

Controller изменения

```
export function updateProduct(req, res) {  
  const id = Number(req.params.id);  
  const product = updateProductById(id, req.body);  
  
  if (!product) {  
    return res.status(404).json({  
      message: 'Product not found'  
    });  
  }  
  
  res.json(product);  
}
```

Что важно

- id берётся из req.params.
- Изменения берутся из req.body.
- Service выполняет обновление.
- При отсутствии товара возвращается 404.
- При успехе возвращается обновлённый объект.

Controller удаления

```
export function deleteProduct(req, res) {  
  const id = Number(req.params.id);  
  const product = deleteProductById(id);  
  
  if (!product) {  
    return res.status(404).json({  
      message: 'Product not found'  
    });  
  }  
  
  res.json(product);  
}
```

Что важно

- Controller получает id удаляемого товара.
- Service выполняет удаление.
- Если товар отсутствует, возвращается 404.
- В учебном API успешный ответ содержит удалённый объект.

Полный CRUD-цикл товара

- 1 Create** `POST /api/products` создаёт новый товар и возвращает его с новым id.
- 2 Read** `GET /api/products/:id` позволяет получить созданный товар.
- 3 Update** `PATCH /api/products/:id` изменяет выбранные свойства товара.
- 4 Read after Update** Повторный GET позволяет убедиться, что изменения сохранены.
- 5 Delete** `DELETE /api/products/:id` удаляет товар.
- 6 Read after Delete** Повторный GET для удалённого id должен вернуть 404.



Учебный вопрос 5. Ответы REST API

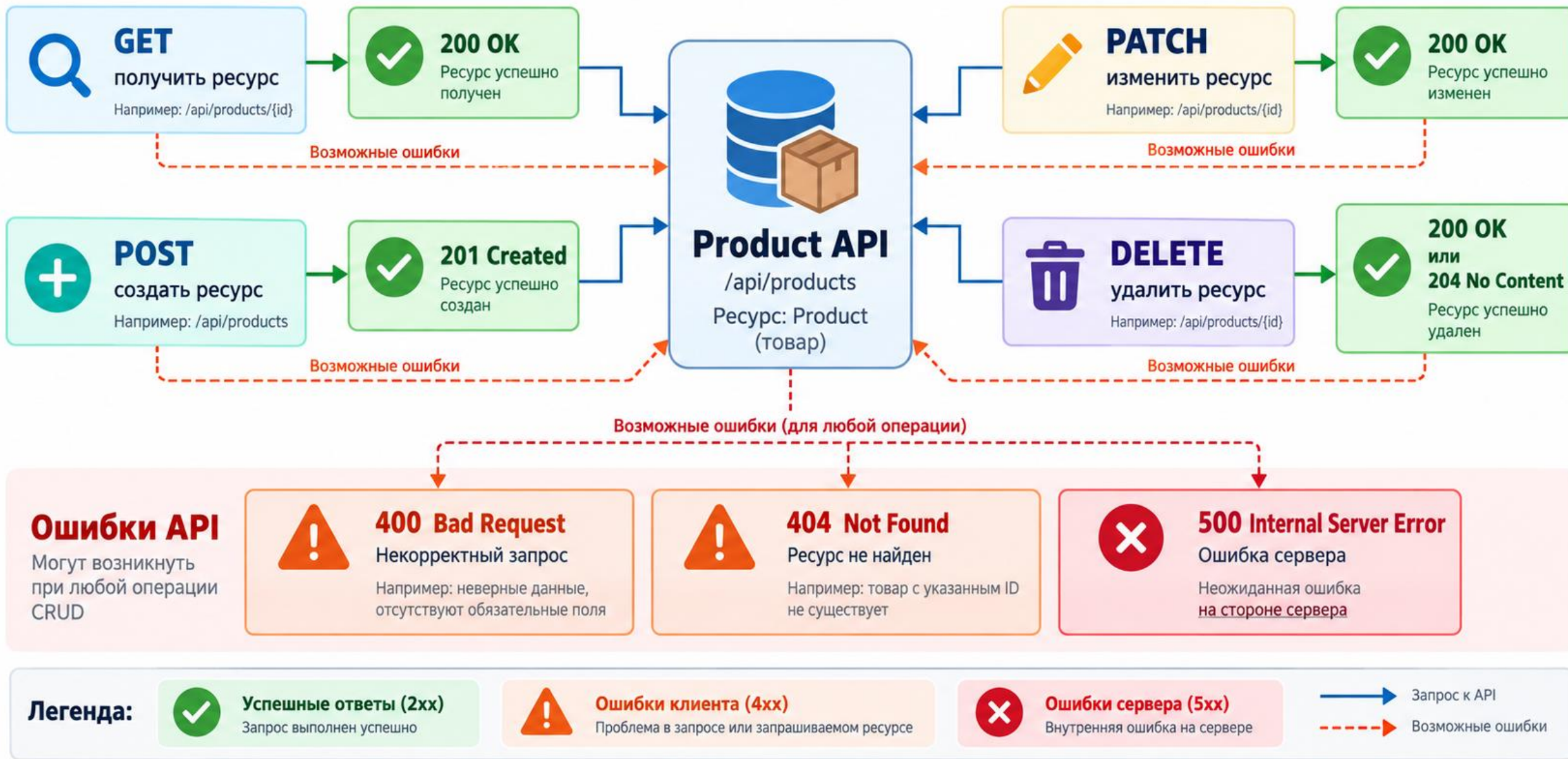
Статусы HTTP помогают клиенту понять результат операции

Основные HTTP-коды Product API

Код	Название	Когда используем
200	OK	Успешное получение или изменение с телом ответа
201	Created	Новый ресурс успешно создан
204	No Content	Операция выполнена, тело ответа не требуется
400	Bad Request	Запрос содержит некорректные данные
404	Not Found	Запрошенный ресурс не существует
500	Internal Server Error	Ошибка на стороне сервера

HTTP-статусы полного CRUD API

Один ресурс — четыре операции — понятные ответы сервера



HTTP-код и JSON выполняют разные задачи

HTTP-код

- Показывает общий результат запроса
- 200 — операция успешна
- 201 — ресурс создан
- 404 — ресурс не найден
- Позволяет frontend быстро определить тип результата

JSON

- Передаёт данные результата
- Может содержать созданный или изменённый объект
- Может содержать сообщение об ошибке
- Используется JavaScript-кодом frontend
- Структура определяется нашим API

Мини-практика: проверьте CRUD API

Выполните полный жизненный цикл одного товара через HTTP-клиент.

Задание

- Создать товар через POST /api/products
- Запомнить полученный id
- Получить товар через GET /api/products/:id
- Изменить цену через PATCH /api/products/:id
- Повторно получить товар и проверить новую цену
- Удалить товар через DELETE /api/products/:id
- Проверить GET удалённого товара

Критерии проверки

- POST возвращает созданный объект
- GET возвращает нужный товар
- PATCH изменяет только переданные поля
- DELETE удаляет нужный ресурс
- После удаления GET возвращает 404

Контрольные вопросы

1. Что такое ресурс в REST API?
2. Какие четыре операции входят в CRUD?
3. Какой HTTP-метод обычно используется для создания ресурса?
4. Чем отличаются GET /api/products и GET /api/products/15?
5. Почему действие create не требуется добавлять в URL?
6. Чем PATCH отличается от PUT?
7. Какой статус обычно возвращается после успешного создания ресурса?
8. Что означает статус 404?
9. Как распределяется CRUD-логика между Route, Controller и Service?
10. Как проверить полный CRUD-цикл ресурса?

Итоги занятия

1

REST API организует взаимодействие вокруг ресурсов.

2

CRUD включает Create, Read, Update и Delete.

3

HTTP-метод определяет операцию над ресурсом.

4

Коллекция `products` доступна через `/api/products`, отдельный товар — через `/api/products/:id`.

5

Routes определяют endpoints, controllers работают с HTTP, services выполняют операции с данными.

6

HTTP-коды сообщают клиенту результат выполнения запроса.

7

Полный CRUD API позволяет frontend создавать, получать, изменять и удалять данные.